

Master's thesis

Master's Programme in Computer Science

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Ha Vien

August 18, 2026

Faculty of Science

University of Helsinki

Contact information

P. O. Box 68 (Pietari Kalmin katu 5)
00014 University of Helsinki,Finland

Email address: info@cs.helsinki.fi
URL: http://www.cs.helsinki.fi/

HELSINGIN YLIOPISTO – HELSINGFORS UNIVERSITET – UNIVERSITY OF HELSINKI

Tiedekunta — Fakultet — Faculty	Koulutusohjelma — Utbildningsprogram — Study programme
Faculty of Science	Master's Programme in Computer Science

Tekijä — Författare — Author

Ha Vien

Työn nimi — Arbetets titel — Title

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Ohjaajat — Handledare — Supervisors

Keijo Heljanko, Christoph Luneburg, Petteri Kotimaa

Työn laji — Arbetets art — Level	Aika — Datum — Month and year	Sivumäärä — Sidoantal — Number of pages
Master's thesis	August 18, 2026	23 pages

Tiivistelmä — Referat — Abstract

Abstract to be written last.

ACM Computing Classification System (CCS)
Software and its engineering → Software notations and tools → General programming languages
Computer systems organization → Real-time systems → Real-time system specification
Networks → Network types → Mobile networks

Avainsanat — Nyckelord — Keywords

Rust, C++, RAN control plane, tail latency, memory safety, fault injection, incremental adoption

Säilytyspaikka — Förvaringsställe — Where deposited

Helsinki University Library

Muita tietoja — övriga uppgifter — Additional information

Software study track

Contents

1 Introduction 1

2 Background: Systems Languages for Latency-Critical Software 3

3 Problem Context: UE Context Management and Predictability 5

4 Objectives 7

5 Research Methodology 9

6 Design and Implementation 11

7 Experimental Setup and Evaluation 13

8 Discussion 15

9 Conclusion and Future Work 17

Statement on the Use of AI Tools 19

Bibliography 21

1 Introduction

- The 5G RAN control plane (the gNB-CU-CP) now runs as software on ordinary servers. It is large, it keeps state for every connected device, it serves many requests at once, and the 5G target for control plane latency, the transition from a battery-efficient state to continuous data transfer, is 10 ms [1]. One defect, or one regression in the slowest requests, therefore degrades the whole site. The work is carried out at Nokia on a UE Context Management service modelled on a real gNB-CU-CP component.

- This software is written mostly in C++. C++ gives the control over memory and timing that such a system needs, but it leaves memory-safety and concurrency mistakes to the programmer [2]. Rust promises the same performance while rejecting those mistakes at compile time [3]. The choice has a long horizon: 3GPP has allocated Release 20 to the 6G studies and Release 21 to the normative work, with migration from 5G in scope [4], so a language chosen for the control plane of the current generation is inherited by the next.

- The problem is a lack of evidence. Telecom vendors are under growing pressure to move to memory-safe languages [5], and the closest prior work is a cluster of at least five Ericsson master's theses, none of which answers the question as posed here. Three keep the runtime in play alongside the language, comparing Boost.Fibers, goroutines, Tokio, Monoio, Glommio and DPDK Eventdev rather than the languages that host them [6, 7, 8]. The other two report execution time and peak memory rather than tail behaviour under rising load [9, 10]. Emberg and Wadsten come closest in method, with a matched C++ and Rust pair joined by a C wrapper, but measure a stateless transformation [10]. No study holds the specification, concurrency model, and build settings fixed for a stateful, concurrent subsystem while sweeping past saturation and measuring incremental migration cost.

- That evidence is also concentrated in one vendor and untested outside it, while on the other side there is no study of Rust among Nokia's indexed publications at all. The nearest peer-reviewed position from that side argues for making

memory-error sanitisation practical rather than for replacing it with a compile-time guarantee [11], which is the strategy this thesis tests against.

- Four research questions frame the work. RQ1 compares latency predictability (median, p99, p99.9, maximum) under increasing load and overload with the concurrency model held fixed [12]. RQ2 asks which defect classes Rust removes at compile time, at what stage the rest are detected, and which classes it does not prevent. RQ3 asks what overhead Rust's safety model imposes on throughput, CPU, and memory. RQ4 asks what incremental adoption costs in effort, FFI interoperability, and risk inside an existing C++ RAN codebase.

- The work contributes five things: a reproducible, telecom-relevant benchmarking framework; a frozen shared specification with two parity-tested implementations; a quantitative comparison that emphasises tail latency under overload; a defect-class taxonomy that marks where Rust's compile-time guarantee stops; and evidence-based guidelines for incremental adoption, where the dominant cost is the learning curve rather than run-time [13].

2 Background: Systems Languages for Latency-Critical Software

- In the NG-RAN architecture the base station (gNB) is split into a central and a distributed unit, and the central unit is split again into a control plane and a user plane. UE Context Management sits in the control plane, the gNB-CU-CP, and reaches its neighbours over the standardised F1, E1, and NG interfaces [14].
- Deadlines are met or missed because of predictability, not average speed. The metrics that matter are therefore the p99 and p99.9 latencies and the jitter around them [12]. They have to be measured past the saturation point and recorded without coordinated omission, that is, without letting a load generator that has fallen behind hide its own slowest requests [15, 16].
- C++ offers abstractions that cost nothing at run time, deterministic cleanup through RAII, and no garbage collector. Safety, however, remains the programmer's responsibility [2], and sanitizers report only the errors that a given test run happens to reach.
- Rust has the same run-time model and also no garbage collector, but it checks memory safety and freedom from data races at compile time, using ownership, borrowing, and thread-safety traits such as `Send` [3, 17]. Code that needs to step outside those rules must be marked `unsafe`, which keeps such code small and easy to find.
- The comparison is therefore C++ against Rust. Garbage-collected languages are left out because their pauses are incompatible with the latency targets. The concurrency model is held fixed in both implementations, and the C ABI is treated as the realistic adoption path. No earlier study isolates the language this way for a stateful, concurrent subsystem [6, 10].

3 Problem Context: UE Context Management and Predictability

- UE Context Management owns the lifecycle of one context per connected device. Each context moves through four states, Idle, Connected, Reconfiguring, and Released, driven by a small set of lifecycle operations. An operation that is illegal in the current state returns a typed error instead of being carried out.
- The subsystem was chosen because it is both representative and small enough to keep under control. It has mutable state per device, a strict lifecycle, heavy concurrency, and hard latency targets [1, 18], and it is still simple enough to specify precisely and to test for identical behaviour in two languages.
- Five defect classes fall inside Rust's guarantee, and each has a natural form in this subsystem: use-after-free, double-free, iterator invalidation, data race, and unguarded shared mutation. What the guarantee rests on is aliasing control and data-race freedom [17]. The catalogue is this study's own.
- Five further classes are included on purpose from outside that guarantee: a lock-order deadlock, a lost update, a leak through a reference cycle, an integer overflow in a release build, and the blast radius of a failed shard worker. They keep the study from confirming a foregone conclusion and show instead where the guarantee stops. Two of them return the opposite of the expected result, with the C++ tooling reporting what Rust's does not.
- Scale is what makes the tail matter. One procedure triggers many operations and one site serves many devices, so a p99.9 measured on a single operation becomes a steady rate of degraded procedures in the field [12]. This is why the load sweep is pushed past the point where the service starts to fall behind.

4 Objectives

- A documented literature search of the citation databases and of the Nordic thesis repositories, where the closest work turns out to live, found no study that holds a stateful control-plane subsystem fixed while measuring tail latency of Rust and C++ past saturation under the same concurrency model [6, 8, 10].
- Five objectives follow from the research questions: measure predictability (O1), establish which defect classes are prevented (O2), measure the resource overhead (O3), build and measure a migration prototype (O4), and publish reusable artifacts and guidance (O5).
- Three hypotheses are stated in advance, so that the measurements can refute them. H1: below saturation the two implementations perform the same. The prior evidence points to near-parity with a measurable residual rather than to equality, a Rust network driver running 4% slower than its C reference while executing 63% more instructions per packet [19]. H2: Rust prevents at compile time the defect classes for which C++ can only offer best-effort detection. H3: the cost of calling Rust from C++ is negligible.
- The work succeeds if the evidence is good enough to base a decision on: repeated runs reported with confidence intervals [20], defect claims shown on programs that really compile and run, migration cost read off a working prototype, and every number reproducible from the published artifacts.
- The scope is drawn narrowly to protect internal validity. The full gNB, the PHY and L2 layers, the user plane, captured traffic traces, managed languages, and production operations are all left out.

5 Research Methodology

- The approach is empirical and constructive. Instead of comparing two systems that already exist and differ in many ways, the study builds a matched pair of implementations from one frozen specification, so that the language is the only systematic difference between them. Both use an identical thread-per-shard concurrency model, because that structuring itself changes tail latency [21].

- RQ1 and RQ3 are answered with an open-loop load sweep that runs from light load into overload on pinned cores. Open-loop means the generator keeps to its send schedule instead of waiting for replies. Latency is recorded in identically configured HDR histograms [16], and resource use is sampled through the operating system. The results so far come from a dedicated dual-socket Xeon host running as a KVM guest. That host resolves the overload divergence and the resource comparison, but not the far tail below saturation: the frequency governor is not observable from inside the guest and CPU steal is non-zero. Locking frequency requires the bare-metal campaign.

- RQ2 is answered by controlled fault injection [22]. A fixed catalogue of defects is written in both languages, and the expected outcome of each one is recorded in advance, so that a surprise shows up as a discrepancy instead of being explained away afterwards.

- RQ4 is answered by a working prototype in which C++ drives the Rust core through a foreign-function interface, the seam being where cross-language memory-management defects concentrate [23]. It yields three measurements: how large the seam is, how much `unsafe` code it needs, and what one call across it costs.

- Fairness is the central experimental control: the specification is frozen, allocator and optimisation settings are matched, and every point where the two languages had to differ is written down in a forced-idiom log. Reporting follows the guidance that a single figure invites conclusions run-to-run variation does not support [20]: medians with nonparametric bootstrap confidence intervals [24], and exact paired

permutation tests corrected for multiple comparisons by the Holm procedure [25]. All data is synthetic, so no data-protection concerns arise.

6 Design and Implementation

7 Experimental Setup and Evaluation

8 Discussion

9 Conclusion and Future Work

Statement on the Use of AI Tools

Tool. Anthropic's Claude Opus 5, used through the Cursor development environment, is the only generative AI system used in the preparation of this thesis. It was used throughout the work, from June to August 2026. Following University of Helsinki policy the model is neither named as an author nor cited as a source.

What the assistance covered. It was extensive, and is declared as such. The model drafted and revised the running prose of this thesis and proposed the order in which its arguments are made. It wrote and refactored the two implementations of the UE Context Management service, the load generator, the fault-injection catalogue, and the interoperability prototype. It wrote the analysis code behind the bootstrap intervals, the exact paired permutation tests, and the Holm correction, together with the figure-generating and build scripts and the LaTeX sources. It also suggested candidate literature. The abstract is the one part of this thesis to which that assistance must not extend, because the University's thesis guide does not permit a language model in preparing the thesis summary, which in this programme is also the maturity test. In this draft it has not yet been rewritten unaided, and this statement will say so until it has been.

What the model did not settle. The research problem, the four research questions, and the three hypotheses are the author's, as is the control on which the study's value rests: freezing one language-neutral specification and one concurrency model, so that the language remains the only systematic difference between the two implementations. So is the decision that keeps RQ2 falsifiable, which was to carry defect classes from outside Rust's compile-time guarantee so that the premise could be embarrassed rather than merely confirmed. Every measurement reported here was run by the author on the hosts documented in this thesis, and no measured value came from a language model. Each measured number reaches the text through a macro regenerated from the raw logs, so a figure cannot be transcribed by hand, by the author or by the model. The readings of those numbers are the author's as well, including the two restraints the evidence imposes: a far tail a virtualised host cannot resolve is not equivalence, and a sweep dominated by cross-thread handoff cannot bound an effect of the language.

How the output was checked. Model output was treated as a draft to be falsified

rather than as a result, and claims were checked against the sources cited for them. Every bibliography entry was resolved against Crossref, DataCite, or the issuing body, and each was traced to the page or clause carrying the claim made of it. That audit found four passages where the prose overstated or misplaced its source, one of them a sentence inside quotation marks that does not appear in the work it was attributed to. All four were corrected, and the audit is kept with the artefacts so that the correction remains reviewable. Code was checked by running it, against one conformance suite whose transcripts are diffed line by line and a fault catalogue that fixes each expected outcome before the defect is injected. Nothing is asserted here on the model's authority.

What the author took from it. The model is dependable at mechanical work, at transposing an idiom between two languages or keeping a build honest. It is undependable in a specific and predictable way: it will supply a plausible citation for a claim and a confident sentence where the data supports a hedge. Its fluency is not evidence. The effort saved in drafting is spent again, and rightly, in checking, and the parts of this thesis that are defensible are the parts that were checked against a source, a specification, or a measurement.

Responsibility. The author is responsible for the whole of this thesis: its language, its facts, its code, its measurements, and its conclusions. Where it is wrong, the error is the author's.

Bibliography

[1] 3GPP. *Study on scenarios and requirements for next generation access technologies*. Technical Report (TR) 38.913. V18.0.0, Release 18. Clause 7.4 sets the control plane latency target, for the transition from a battery-efficient state to the start of continuous data transfer, at 10 ms. 3rd Generation Partnership Project (3GPP), Apr. 2024. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.913/38913-i00.zip.

[2] L. Szekeres, M. Payer, T. Wei, and D. Song. "SoK: Eternal War in Memory". In: *Proceedings of the 2013 IEEE Symposium on Security and Privacy (S&P)*. 2013, pp. 48–62. DOI: 10.1109/SP.2013.13. URL: https://doi.org/10.1109/SP.2013.13.

[3] S. Klabnik and C. Nichols. *The Rust Programming Language*. 2nd. San Francisco, CA: No Starch Press, 2023. ISBN: 978-1-7185-0310-6.

[4] J. Almodovar and A. Sultan. *The SA1 road to 6G*. 3GPP Highlights, Issue 09. Written by the chair and the secretary of 3GPP WG SA1. Release 20 carries the 6G studies and Release 21 the normative work; ITU-R sets early 2029 for technology proposals and mid-2030 for the submitted specifications. Accessed August 2026. Dec. 2024. URL: https://www.3gpp.org/news-events/3gpp-news/sa1-6g-road.

[5] Cybersecurity and Infrastructure Security Agency. *The Case for Memory Safe Roadmaps: Why Both C-Suite Executives and Technical Experts Need to Take Memory Safe Coding Seriously*. Cybersecurity Information Sheet. CISA, NSA, FBI, ASD's ACSC, CCCS, NCSC-UK, NCSC-NZ, CERT NZ, Dec. 2023. URL: https://www.cisa.gov/resources-tools/resources/case-memory-safe-roadmaps.

[6] R. Karlbäck and A. Orö. "Holistic View on Alternative Programming Languages for Radio Access Network Applications in Cloud and Embedded Deployments". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–2021/023–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2021. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-176307.

[7] J. Olsson and I. Stenström. “Rust Async for Thread-per-Core Packet Processing”. Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–23/052–SE. Master’s thesis. Linköping University, Department of Computer and Information Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-197029.

[8] A. Arvidsson and M. Kaller. “Evaluating Asynchronous Programming Frameworks for 5G Control Plane Procedures”. Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–25/055–SE. Master’s thesis. Linköping University, Department of Computer and Information Science, 2025. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-214746.

[9] V. Olsson. “Rust Programming Language as an Alternative to C for RAN Management Applications”. Carried out at Ericsson AB. Master’s thesis. KTH Royal Institute of Technology, School of Electrical Engineering and Computer Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-325822.

[10] C. Emberg and A. Wadsten. “Attainable Safety and Long-Term Resource-Efficiency Using Rust: Evaluating the Viability of Rust in Terms of Development Effort and Performance for the Decompilation of ASN.1 Messages”. Carried out at Ericsson, Linköping. Report LIU-IDA/LITH-EX-A–23/078–SE. Master’s thesis. Linköping University, Department of Computer and Information Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-195552.

[11] R. Gjomemo, P. H. Phung, E. Ballou, K. S. Namjoshi, V. N. Venkatakrishnan, and L. Zuck. “Leveraging Static Analysis Tools for Improving Usability of Memory Error Sanitization Compilers”. In: *Proceedings of the 2016 IEEE International Conference on Software Quality, Reliability and Security (QRS)*. Two of the six authors are affiliated with Nokia. 2016, pp. 323–334. DOI: 10.1109/QRS.2016.44. URL: https://doi.org/10.1109/QRS.2016.44.

[12] J. Dean and L. A. Barroso. “The Tail at Scale”. In: *Communications of the ACM* 56.2 (2013), pp. 74–80. DOI: 10.1145/2408776.2408794. URL: https://doi.org/10.1145/2408776.2408794.

[13] K. R. Fulton, A. Chan, D. Votipka, M. Hicks, and M. L. Mazurek. “Benefits and Drawbacks of Adopting a Secure Programming Language: Rust as a Case Study”. In: *Proceedings of the 17th Symposium on Usable Privacy and Security (SOUPS)*. USENIX Association, 2021, pp. 597–616. URL: https://www.usenix.org/conference/soups2021/presentation/fulton.

[14] 3GPP. *NG-RAN; Architecture description*. Technical Specification (TS) 38.401. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), June 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.401/38401-h50.zip.

[15] B. Schroeder, A. Wierman, and M. Harchol-Balter. "Open Versus Closed: A Cautionary Tale". In: *Proceedings of the 3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '06)*. 2006. URL: https://www.usenix.org/conference/nsdi-06/open-versus-closed-cautionary-tale.

[16] G. Tene. *HdrHistogram: A High Dynamic Range Histogram*. Project documentation and reference implementation. Section "Corrected vs. raw value recording calls" documents the `recordValueWithExpectedInterval` correction applied when a response time exceeds the expected interval between requests. Accessed August 2026. URL: https://github.com/HdrHistogram/HdrHistogram.

[17] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. "Safe Systems Programming in Rust". In: *Communications of the ACM* 64.4 (2021), pp. 144–152. DOI: 10.1145/3418295. URL: https://doi.org/10.1145/3418295.

[18] 3GPP. *Service requirements for the 5G system; Stage 1*. Technical Specification (TS) 22.261. V18.6.0, Release 18. 3rd Generation Partnership Project (3GPP), Mar. 2022. URL: https://www.3gpp.org/ftp/Specs/archive/22_series/22.261/22261-i60.zip.

[19] P. Emmerich, S. Ellmann, F. Bonk, A. Egger, E. García Sánchez-Torija, T. Günzel, S. Di Luzio, A. Obada, M. Stadlmeier, S. Voit, and G. Carle. "The Case for Writing Network Drivers in High-Level Programming Languages". In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2019, pp. 1–13. DOI: 10.1109/ANCS.2019.8901892. URL: https://doi.org/10.1109/ANCS.2019.8901892.

[20] A. Georges, D. Buytaert, and L. Eeckhout. "Statistically Rigorous Java Performance Evaluation". In: *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*. 2007, pp. 57–76. DOI: 10.1145/1297027.1297033. URL: https://doi.org/10.1145/1297027.1297033.

[21] P. Enberg, A. Rao, and S. Tarkoma. "The Impact of Thread-per-Core Architecture on Application Tail Latency". In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2019, pp. 1–8. DOI: 10.1109/ANCS.2019.8901874. URL: https://doi.org/10.1109/ANCS.2019.8901874.

[22] R. Natella, D. Cotroneo, and H. S. Madeira. "Assessing Dependability with Software Fault Injection: A Survey". In: *ACM Computing Surveys* 48.3 (2016), 44:1–44:55. DOI: 10.1145/2841425. URL: https://doi.org/10.1145/2841425.

[23] Z. Li, J. Wang, M. Sun, and J. C. S. Lui. "Detecting Cross-Language Memory Management Issues in Rust". In: *Computer Security – ESORICS 2022*. 2022, pp. 680–700. DOI: 10.1007/978-3-031-17143-7_33. URL: https://doi.org/10.1007/978-3-031-17143-7_33.

[24] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993. ISBN: 978-0-412-04231-7. DOI: 10.1007/978-1-4899-4541-9. URL: https://doi.org/10.1007/978-1-4899-4541-9.

[25] S. Holm. "A Simple Sequentially Rejective Multiple Test Procedure". In: *Scandinavian Journal of Statistics* 6 (1979), pp. 65–70. URL: https://www.jstor.org/stable/4615733.